

Part 2: Logical Database Design

Yüksel Ege Boyacı – 2023400315

Tuğçe Yücel – 2022400105

Contents

1	Relational Schema (SQL DDL)	3
2	Constraints Not Expressible in DDL	9

1 Relational Schema (SQL DDL)

The logical database design transforms our conceptual model into a concrete relational schema using standard SQL Data Definition Language (DDL). The script below accurately maps all entities, handles normalization for disjoint IS-A relationships across **Person**, implements single and multi-valued mapping, and strictly enforces valid domain structures using **CHECK** constraints.

```
1  -- =====
2  -- TransferDB - Logical Database Design (SQL DDL)
3  -- CMPE 321, Introduction to Database Systems, Spring 2026
4  -- =====
5  -- MySQL Server Syntax
6  -- =====
7  CREATE DATABASE IF NOT EXISTS TransferDB;
8  USE TransferDB;
9  SET FOREIGN_KEY_CHECKS = 0;
10 DROP TABLE IF EXISTS Match_Participation;
11 DROP TABLE IF EXISTS Match;
12 DROP TABLE IF EXISTS Club_Competition_Participation;
13 DROP TABLE IF EXISTS TransferRecord;
14 DROP TABLE IF EXISTS LoanContract;
15 DROP TABLE IF EXISTS PermanentContract;
16 DROP TABLE IF EXISTS Contract;
17 DROP TABLE IF EXISTS Leads;
18 DROP TABLE IF EXISTS Referee;
19 DROP TABLE IF EXISTS Manager;
20 DROP TABLE IF EXISTS Player;
21 DROP TABLE IF EXISTS Club;
22 DROP TABLE IF EXISTS Stadium;
23 DROP TABLE IF EXISTS Competition;
24 DROP TABLE IF EXISTS Person;
25 SET FOREIGN_KEY_CHECKS = 1;
26 -- =====
27 -- 1. Person
28 -- =====
29 -- NOTE: Disjoint, Total ISA: each Person must be exactly one of
30 -- Player, Manager, or Referee. The application must enforce this.
31 CREATE TABLE Person(
32     person_ID INT PRIMARY KEY,
33     name VARCHAR(100) NOT NULL,
34     surname VARCHAR(100) NOT NULL,
35     nationality VARCHAR(100) NOT NULL,
36     date_of_birth DATE NOT NULL,
37     CHECK (CHAR_LENGTH(TRIM(name)) > 0),
38     CHECK (CHAR_LENGTH(TRIM(surname)) > 0),
39     CHECK (CHAR_LENGTH(TRIM(nationality)) > 0)
40 );
41 -- =====
42 -- 2. Player (ISA Person)
43 -- =====
44 CREATE TABLE Player(
45     person_ID INT PRIMARY KEY,
46     market_value DECIMAL(15, 2) NOT NULL,
47     main_position ENUM(
48         'Goalkeeper',
```

```

49         'Defender',
50         'Midfielder',
51         'Forward'
52     ) NOT NULL,
53     strong_foot ENUM('Right', 'Left', 'Both') NOT NULL,
54     height_cm INT NOT NULL,
55     FOREIGN KEY (person_ID) REFERENCES Person(person_ID),
56     CHECK (market_value > 0),
57     CHECK (height_cm > 0)
58 );
59 -- =====
60 -- 3. Manager (ISA Person)
61 -- =====
62 CREATE TABLE Manager(
63     person_ID INT PRIMARY KEY,
64     preferred_formation VARCHAR(20) NOT NULL,
65     experience_level VARCHAR(50) NOT NULL,
66     FOREIGN KEY (person_ID) REFERENCES Person(person_ID),
67     CHECK (CHAR_LENGTH(TRIM(preferred_formation)) > 0),
68     CHECK (CHAR_LENGTH(TRIM(experience_level)) > 0)
69 );
70 -- =====
71 -- 4. Referee (ISA Person)
72 -- =====
73 CREATE TABLE Referee(
74     person_ID INT PRIMARY KEY,
75     license_level VARCHAR(50) NOT NULL,
76     years_experience INT NOT NULL,
77     FOREIGN KEY (person_ID) REFERENCES Person(person_ID),
78     CHECK (years_experience >= 0),
79     CHECK (CHAR_LENGTH(TRIM(license_level)) > 0)
80 );
81 -- =====
82 -- 5. Stadium
83 -- =====
84 CREATE TABLE Stadium(
85     stadium_ID INT PRIMARY KEY,
86     stadium_name VARCHAR(150) NOT NULL,
87     city VARCHAR(100) NOT NULL,
88     capacity INT NOT NULL,
89     UNIQUE (stadium_name, city),
90     CHECK (CHAR_LENGTH(TRIM(stadium_name)) > 0),
91     CHECK (CHAR_LENGTH(TRIM(city)) > 0),
92     CHECK (capacity > 0)
93 );
94 -- =====
95 -- 6. Club
96 -- =====
97 CREATE TABLE Club(
98     club_ID INT PRIMARY KEY,
99     club_name VARCHAR(150) NOT NULL UNIQUE,
100     stadium_ID INT NOT NULL,
101     city VARCHAR(100) NOT NULL,
102     foundation_year INT NOT NULL,
103     FOREIGN KEY (stadium_ID) REFERENCES Stadium(stadium_ID),
104     CHECK (CHAR_LENGTH(TRIM(club_name)) > 0),
105     CHECK (CHAR_LENGTH(TRIM(city)) > 0)
106 );

```

```

107 -- =====
108 -- 7. Leads (Manager <-> Club)
109 -- =====
110 -- NOTE: Manager/Club temporal exclusivity (no overlapping date
111 -- intervals) cannot be fully enforced via CHECK constraints.
112 CREATE TABLE Leads(
113     club_ID INT NOT NULL,
114     manager_ID INT NOT NULL,
115     start_date DATE NOT NULL,
116     end_date DATE NULL,
117     PRIMARY KEY (club_ID, manager_ID, start_date),
118     FOREIGN KEY (club_ID) REFERENCES Club(club_ID),
119     FOREIGN KEY (manager_ID) REFERENCES Manager(person_ID),
120     CHECK (
121         end_date IS NULL
122         OR end_date > start_date
123     )
124 );
125 -- =====
126 -- 8. Contract (Permanent & Loan)
127 -- =====
128 -- NOTE: A player must not have two active contracts of the same
129 -- type concurrently. A Loan contract is only valid if the player
130 -- holds an active Permanent contract with a different parent club.
131 -- These cross-table/temporal constraints aren't checkable here.
132 CREATE TABLE Contract(
133     contract_id INT PRIMARY KEY,
134     player_id INT NOT NULL,
135     club_id INT NOT NULL,
136     start_date DATE NOT NULL,
137     end_date DATE NOT NULL,
138     weekly_wage DECIMAL(15, 2) NOT NULL,
139     contract_type ENUM('Permanent', 'Loan') NOT NULL,
140     FOREIGN KEY (player_id) REFERENCES Player(person_ID),
141     FOREIGN KEY (club_id) REFERENCES Club(club_ID),
142     CHECK (end_date > start_date),
143     CHECK (weekly_wage > 0)
144 );
145 CREATE TABLE PermanentContract(
146     contract_id INT PRIMARY KEY,
147     FOREIGN KEY (contract_id) REFERENCES Contract(contract_id) ON
        DELETE CASCADE
148 );
149 CREATE TABLE LoanContract(
150     contract_id INT PRIMARY KEY,
151     parent_permanent_contract_id INT NOT NULL,
152     FOREIGN KEY (contract_id) REFERENCES Contract(contract_id) ON
        DELETE CASCADE,
153     FOREIGN KEY (parent_permanent_contract_id) REFERENCES
        PermanentContract(contract_id),
154     CHECK (contract_id <> parent_permanent_contract_id)
155 );
156 -- =====
157 -- 9. Transfer Record
158 -- =====
159 -- NOTE: For a Loan transfer, from_club_id must match the player's
160 -- current parent club and they must have an active permanent contract.
161 CREATE TABLE TransferRecord(

```

```

162     transfer_id INT PRIMARY KEY,
163     player_id INT NOT NULL,
164     from_club_id INT NOT NULL,
165     to_club_id INT NOT NULL,
166     transfer_date DATE NOT NULL,
167     transfer_fee DECIMAL(15, 2) NOT NULL,
168     transfer_type ENUM('Free', 'Purchase', 'Loan') NOT NULL,
169     FOREIGN KEY (player_id) REFERENCES Player(person_ID),
170     FOREIGN KEY (from_club_id) REFERENCES Club(club_ID),
171     FOREIGN KEY (to_club_id) REFERENCES Club(club_ID),
172     CHECK (from_club_id <> to_club_id),
173     CHECK (transfer_fee >= 0),
174     CHECK (
175         (
176             transfer_type = 'Free'
177             AND transfer_fee = 0
178         )
179         OR (
180             transfer_type IN ('Purchase', 'Loan')
181             AND transfer_fee > 0
182         )
183     );
184 );
185 -- =====
186 -- 10. Competition
187 -- =====
188 CREATE TABLE Competition(
189     competition_ID INT PRIMARY KEY,
190     name VARCHAR(150) NOT NULL,
191     season VARCHAR(20) NOT NULL,
192     country VARCHAR(100) NOT NULL,
193     competition_type VARCHAR(50) NOT NULL,
194     UNIQUE (name, season),
195     CHECK (CHAR_LENGTH(TRIM(name)) > 0),
196     CHECK (CHAR_LENGTH(TRIM(country)) > 0)
197 );
198 -- =====
199 -- 11. Club_Competition_Participation
200 -- =====
201 CREATE TABLE Club_Competition_Participation(
202     club_ID INT NOT NULL,
203     competition_ID INT NOT NULL,
204     PRIMARY KEY (club_ID, competition_ID),
205     FOREIGN KEY (club_ID) REFERENCES Club(club_ID),
206     FOREIGN KEY (competition_ID) REFERENCES Competition(competition_ID)
207 );
208 -- =====
209 -- 12. Match
210 -- =====
211 -- NOTE: Scheduling capabilities (no two matches at same stadium within
212 -- 120 mins, no club playing twice within 120 mins) can't be CHECKed.
213 -- NOTE: Match attendance cannot exceed Stadium capacity.
214 -- NOTE: Match result is derived from goals, not stored.
215 CREATE TABLE Match(
216     match_ID INT PRIMARY KEY,
217     match_datetime DATETIME NOT NULL,
218     attendance INT NOT NULL,
219     home_goals INT NOT NULL,

```

```

220     away_goals INT NOT NULL,
221     home_club_ID INT NOT NULL,
222     away_club_ID INT NOT NULL,
223     referee_ID INT NOT NULL,
224     stadium_ID INT NOT NULL,
225     competition_ID INT NOT NULL,
226     FOREIGN KEY (home_club_ID) REFERENCES Club(club_ID),
227     FOREIGN KEY (away_club_ID) REFERENCES Club(club_ID),
228     FOREIGN KEY (referee_ID) REFERENCES Referee(person_ID),
229     FOREIGN KEY (stadium_ID) REFERENCES Stadium(stadium_ID),
230     FOREIGN KEY (competition_ID) REFERENCES Competition(competition_ID)
231 ,
232     CHECK (home_club_ID <> away_club_ID),
233     CHECK (attendance >= 0),
234     CHECK (home_goals >= 0),
235     CHECK (away_goals >= 0)
236 );
237 -- =====
238 -- 13. Match Participation & Stats
239 -- =====
240 -- NOTE: Max 11 starters and 23 total squad players limit per club
241 -- per match cannot be enforced here.
242 -- NOTE: Red/Yellow card suspensions must be handled via application
243 -- logic.
244 CREATE TABLE Match_Participation(
245     match_ID INT NOT NULL,
246     player_id INT NOT NULL,
247     is_starter BOOLEAN NOT NULL,
248     minutes_played INT NOT NULL,
249     position_in_match VARCHAR(30) NOT NULL,
250     goals INT NOT NULL DEFAULT 0,
251     assists INT NOT NULL DEFAULT 0,
252     yellow_cards INT NOT NULL DEFAULT 0,
253     red_cards INT NOT NULL DEFAULT 0,
254     rating DECIMAL(4, 2) NULL,
255     PRIMARY KEY (match_ID, player_id),
256     FOREIGN KEY (match_ID) REFERENCES Match(match_ID),
257     FOREIGN KEY (player_id) REFERENCES Player(person_ID),
258     CHECK (
259         minutes_played >= 0
260         AND minutes_played <= 120
261     ),
262     CHECK (goals >= 0),
263     CHECK (assists >= 0),
264     CHECK (
265         yellow_cards >= 0
266         AND yellow_cards <= 2
267     ),
268     CHECK (
269         red_cards >= 0
270         AND red_cards <= 1
271     ),
272     CHECK (
273         rating IS NULL
274         OR (
275             rating >= 1.0
276             AND rating <= 10.0
277         )
278     )

```

```
276 )  
277 );
```

Listing 1: Logical Schema DDL (part2.sql)

2 Constraints Not Expressible in DDL

Various temporal, historical, cross-table aggregative, and dynamically derived business rules cannot be securely restricted within the inline **CHECK**, **UNIQUE**, **PRIMARY KEY**, or **FOREIGN KEY** DDL functions (which strictly limit reference scope to deterministic evaluation within single rows). Therefore, the following constraints are intended to be enforced via application logic or database triggers:

1. Person specialization total+disjoint (exactly one of Player/Manager/Referee) requires application logic.
2. Manager/Club temporal exclusivity in Leads (no overlapping date intervals).
3. No overlapping same-type active contracts for a player concurrently.
4. Match attendance must not exceed the capacity of the stadium where the match was played.
5. No two matches can be scheduled at the same stadium within 120 minutes of each other.
6. No club (home or away) can appear in two matches whose start times are within 120 minutes of each other.
7. No referee can officiate two matches whose start times are within 120 minutes of each other.
8. At most 11 players with `is_starter=TRUE` per club per match in `Match_Participation`.
9. At most 23 players total per club per match in `Match_Participation`.
10. A player cannot appear in the same match as both a home-team and away-team participant.
11. Person `date_of_birth` must be strictly less than the current date (requires triggers in MySQL).
12. Club `foundation_year` must be strictly less than the current year (requires triggers in MySQL).
13. Two yellow cards in one match should automatically set `red_cards` to true.
14. Suspension rules (Red card = 1 match suspension, 5 yellows = 1 match suspension) require historical querying.
15. Match Result (Win/Loss/Draw) is dynamically derived from home/away goals, rather than explicitly stored as an independent attribute.